

Project3 矩阵乘法优化

目录

Project3 矩阵乘法优化

1. 摘要
2. 实验设计
 - 2.1 实验环境
 - 2.2 实验准备
 - 2.3 数据严谨性
3. 优化过程
 - 3.1 朴素矩阵乘法
 - 3.2 ikj循环优化
 - 3.2.1 原理
 - 3.2.2 结果分析
 - 3.3 Strassen算法
 - 3.3.1 原理
 - 3.3.2 代码实现
 - 3.3.3 结果分析
 - 3.4 SIMD与OpenMP
 - 3.4.1 SIMD应用
 - 3.4.2 三种OpenMP实现
 - 3.4.3 结果分析
 - 3.4.4 原因分析
 - 3.5 内存对齐
 - 3.5.1 背景与原理
 - 3.5.2 代码实现
 - 3.5.3 结果分析与原因
 - 3.6 Out-of-Core
 - 3.6.1 N=64k 遇到的问题
 - 3.6.2 选择OOC的原因
 - 3.6.3 实现细节
 - 3.6.4 结果分析
 - 3.7 CUDA 实现
 - 3.7.1 知识储备
 - 3.7.2 代码实现
 - 3.7.3 结果分析
4. 有关OPENBLAS矩阵乘法的分析
 - 4.1 Goto方法下面向多级缓存的分块
 - 4.2 汇编级优化
5. 矩阵乘法计算误差分析
 - 5.1 结果分析
 - 5.2 原因探究
6. 其他系统与架构下运行结果
 - 6.1 WSL
 - 6.1.1 运行
 - 6.1.2 Valgrind
 - 6.1.3 Perf
 - 6.2 ARM64架构
7. 结论
 - 7.1 主要发现与收获
 - 7.2 算法外限制性能的主要因素

1. 摘要

本项目主要使用了ikj循环优化、Strassen算法、循环展开、分块缓存、OpenMP多线程、SIMD、内存对齐、Out-of-Core、CUDA等技术来优化单精度浮点数 矩阵乘法的性能，并与朴素矩阵乘法和OpenBLAS库的性能进行了对比。并基于朴素矩阵乘法，对上述优化后的乘法进行了计算结果的误差分析。探究了OPENBLAS的优化机制。比较了上述矩阵乘法在x86-64架构和arm架构的性能差异。

2. 实验设计

2.1 实验环境

配置项	详细信息
设备机器	远程操控 Legion REN7000K-26IRX 台式电脑
处理器	Intel(R) Core(TM) i9-149000HX (2.20 GHz) 24核心 32线程
主频	4000~6000MHz
机带 RAM	32.0 GB (31.6 GB 可用)
系统类型	64 位操作系统, 基于 x64 的处理器 Windows 11 家庭中文版
GCC	(x86_64-win32-seh-rev1, Built by MinGW-Builds project) 15.2.0
GPU0	NVIDIA GeForce RTX 4070 Laptop GPU
CUDA	Version: 12.6
Driver	Version: 561.17
总内存	24193MB
显示内存 (VRAM)	7948MB
共享内存	16245MB
Clocks	Graphics : 1605 MHz SM : 1605 MHz Memory : 8000 MHz Video : 1500 MHz
总CUDA核心数	4608
SM 数量	36
WSL版本	2.6.3.0

配置项	详细信息
Mac mini	Mac-A17
芯片	Apple M4
内存	16GB
macOS	macOS Tahoe 26.4.1
编译器	Apple Clang version 21.0.0
CPU物理核心数	10

2.2 实验准备

windows管理员终端打开指定的文件夹，输入

```
git clone https://github.com/feitianduowen/Proj3CPP.git
mkdir OpenBLAS
```

前往 [sourceforge](#)，下载适用于 Windows 的 OpenBLAS 预编译版本（例如 openBLAS-0.3.23-x64.zip）。解压后，将 lib 目录中的 libopenblas.a、libopenblas.dll.a 和 bin 里的 libopenblas.dll 文件放在 Proj3CPP/OpenBLAS 目录下。

用CLion打开Proj3CPP文件夹，等待CMake加载完成后，可以看到可选的CMake应用程序 test_matmul。

如果使用命令行编译，需要加上 -fopenmp。

CUDA

下载适合电脑版本和NVIDIA GPU 的 CUDA Toolkit（例如 CUDA 12.0）。安装完成后，确保 CUDA 的 bin 目录（如 D:\Program Files\NVIDIA GPU Computing Toolkit\CUDA\v12.0\bin）已添加到系统的环境变量 PATH 中。下载合适的visual studio版本（如 Visual Studio 2022），安装时选择“使用 C++ 的桌面开发”工作负载。

尝试多次，无法在CLion中成功配置CUDA编译环境(很多版本c++、c的编译器以被停用，或者不支持)，最终选择了在Visual Studio code中实现CUDA乘法。

如果vs code的输出窗口显示找不到“cl.exe”，请输入

```
call "your_path_to_Microsoft Visual
Studio\2022\Community\VC\Auxiliary\Build\vcvars64.bat" #需修改为本地的路径
```

进入到 matmul.cu 所在目录，在终端输入

```
nvcc matmul.cu -o matmul.exe -xcompiler "/openmp /arch:AVX2 /utf-8"
-allow-unsupported-compiler #如果上面的失败了，加上这个选项
```

在我的local环境中，只有一个CUDA设备。

2.3 数据严谨性

本项目因为主要探究i性能，因此只考虑边长大小为16倍数的矩阵。所有方法都不适应非16倍数矩阵。且只考虑了两个大小相同的N*N矩阵乘法。

时间测量代码方面使用了 `timespec_get` 函数获取高精度的**现实世界时间戳**，不受底层线程并行的影响。

实际测量时间=物理极限最快时间+系统噪音耗时(`context switch`、硬件中断等)。因此添加内循环和外循环来多次测量同一配置的性能，**取最小值**作为最终结果，以减小偶然因素的影响。内循环测量同一组参数多次，外循环则是重复整个测试过程多次。

现代CPU具有动态睿频(Turbo Boost)和**温度墙**限制，在连续进行多次大矩阵查时，CPU可能会因为过热而降低频率，导致后续测试的性能下降。为了减轻这个问题，我使用 `windows.h` 中的 `sleep`，如果是Linux则使用 `#include <unistd.h> sleep(1);`，在每次测试之间加入了适当的休息时间（如10秒），以允许CPU降温并恢复正常频率。此外，在测试过程中，我使用轻量的软件**Core Temp 1.20** 监控了CPU温度，确保它不会过高。并且在每次运行代码前后间隔较长时间，软件界面如图：



3. 优化过程

3.1 朴素矩阵乘法

作为后续实验的基准。时间复杂度为 $O(N^3)$

3.2 ikj循环优化

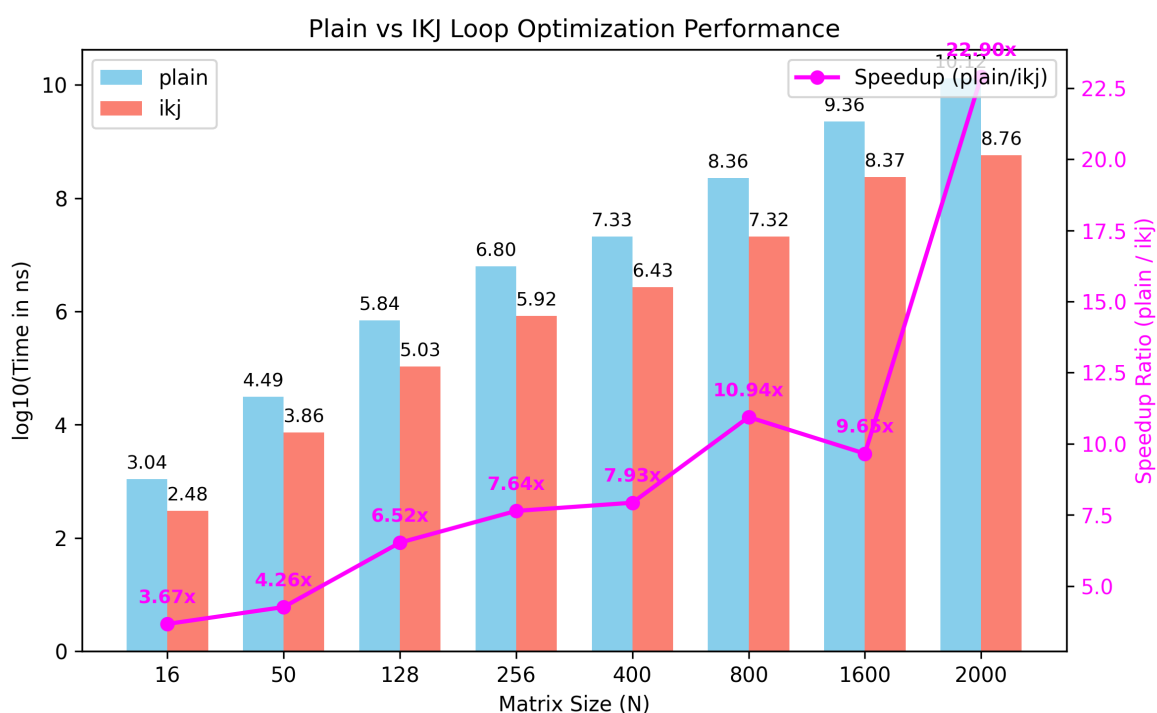
3.2.1 原理

在矩阵运算时，对A的访存是连续的，而对B的访存是跳跃的（strided access）。当矩阵规模较大时，CPU的缓存机制会导致大量的Cache Miss，从而显著降低性能。

根据[某学长的分析](#)¹，各种循环顺序，总的跳转次数中， ikj 的跳转次数最少（ n^2 ），性能最好。

综上，通过调整循环顺序为ikj，我们可以使得对B的访问变为连续访问，从而大幅提升性能。

3.2.2 结果分析



发现ikj的循环优化对规模大的矩阵提升更显著, 符合理论分析中的预期。

3.3 Strassen算法

3.3.1 原理

令：

$$A = \begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix}, \quad B = \begin{pmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{pmatrix}, \quad C = \begin{pmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{pmatrix}$$

其中每个子块都是 $\frac{n}{2} \times \frac{n}{2}$ 的矩阵。

根据矩阵乘法规则：

$$\begin{aligned} C_{11} &= A_{11}B_{11} + A_{12}B_{21} \\ C_{12} &= A_{11}B_{12} + A_{12}B_{22} \\ C_{21} &= A_{21}B_{11} + A_{22}B_{21} \\ C_{22} &= A_{21}B_{12} + A_{22}B_{22} \end{aligned}$$

这需要 **8次** $\frac{n}{2} \times \frac{n}{2}$ 矩阵乘法 and **4次** 矩阵加法。

递归做下去，时间复杂度递推式为：

$$T(n) = 8T\left(\frac{n}{2}\right) + O(n^2)$$

而strassen使用了7个乘积与组合

$$M_1 = (A_{11} + A_{22})(B_{11} + B_{22})$$

$$M_2 = (A_{21} + A_{22})B_{11}$$

$$M_3 = A_{11}(B_{12} - B_{22})$$

$$M_4 = A_{22}(B_{21} - B_{11})$$

$$M_5 = (A_{11} + A_{12})B_{22}$$

$$M_6 = (A_{21} - A_{11})(B_{11} + B_{12})$$

$$M_7 = (A_{12} - A_{22})(B_{21} + B_{22})$$

C的子块可以表示为这些 M_i 的加减。因此复杂度为

$$T(n) = 7T\left(\frac{n}{2}\right) + O(n^2)$$

$$T(n) \approx O(n^{2.807})$$

3.3.2 代码实现

```
void strassen_add(size_t N, const float* A, size_t lda, const float* B, size_t
ldb, float* C, size_t ldc);
void strassen_sub(size_t N, const float* A, size_t lda, const float* B, size_t
ldb, float* C, size_t ldc);
void matmul_ikj_stride(size_t N, const float* A, size_t lda, const float* B,
size_t ldb, float* C, size_t ldc);
void strassen_recursive(size_t N, const float* A, size_t lda, const float* B,
size_t ldb, float* C, size_t ldc);
int matmul_strassen(size_t N, const Matrix *A, const Matrix *B, Matrix const
*C);
```

`matmul_strassen` 是顶层调用接口，调用递归方法 `strassen_recursive`。

在 `strassen_recursive` 方法中当 $N \leq 128$ 时，调用 `matmul_ikj_stride`；大于128则对矩阵分块，并用strassen算法计算分块后的矩阵。使用指针偏移和主维参数来引用子矩阵的起始位置，无需显式拷贝子矩阵。并为七个临时的乘积结果(3.3.1中的 $M_1 \sim M_7$)和两个加减法统一分配缓冲，减少malloc调用次数，防止栈溢出。并对乘积使用递归的调用。计算结束后释放buffer。

在 `matmul_ikj_stride` 方法中，先对目标矩阵中对应的位置清零，再使用ikj循环对子矩阵做乘法计算。

参数lda,ldb,ldc分别是子矩阵A,B,C的主维(leading dimension)

3.3.3 结果分析



根据上图，Strassen算法性能随着矩阵规模的增大而增强，在 $N=8192$ 时，Strassen算法的性能已经远远优于朴素矩阵乘法，远比ikj循环优化的版本还要快。符合理论预期。

3.4 SIMD与OpenMP

3.4.1 SIMD应用

```
for (int i = 0; i < N; i += 4)
{
    for (int j = 0; j < N; j += 8)
    {
        __m256 c0 = _mm256_loadu_ps(&c->data[(i + 0) * N + j]);
```

对于形如上面的代码：__m256 是一种数据类型，代表一个 256 位的AVX向量寄存器，装了矩阵 C 中第 i 行、从第 j 列(本小节出现的 j 都是 8 的倍数)开始的连续 8 个 float。

将 B 的 k_{th} 行 j_{th} 列的 8 个连续的 float 分别与 A 的 i_{th} 行 k_{th} 列的八个连续的 float 相乘，存入 C 的 i_{th} 行 j_{th} 列开始的 8 个连续的 float 中。

3.4.2 三种OpenMP实现

因为是在实验过程中，是先测试的SIMD和OpenMP，再发现并探究的Strassen算法。因此 `matmul_improved2` 没有使用Strassen，而 `matmul_improved` 使用了Strassen。

方案1: 未引入STRASSEN方法前-- `matmul_improved2`

当矩阵规模小于128时，不使用分块和OpenMP，直接循环展开和SIMD。

当矩阵规模大于128时，使用分块缓存，块大小为128*128。使用 OpenMP 对拆分后的 Block 进行多线程并行。具体实现代码为：

```
#pragma omp parallel for collapse(2) default(none) shared(N, A, B, C, BLOCK)
```

#pragam omp 激活OpenMP功能。parallel for 将for循环的迭代工作划分给并行区域(parallel 创建的)中的线程去执行，每个线程处理一部分迭代，所有线程完成后合并

`collapse(2)` 将后面的嵌套的两层循环合并为一个大的迭代空间，再分割给各线程，使负载更均衡。

⚠️⚠️⚠️ 但是在本方案中，计算 C_i 的双层嵌套循环 `for` 和 `for` 没有紧挨(非完美嵌套)，所以在 Windows MinGw 下 `collapse(2)` 失效，在 WSL 或者 Linux 下会报错，因为 Linux 系统上的 GCC 对此更严格。

因为 `taskwait` 被注释掉，主线程在分配完 P1~P7 的 `task` 后，**不会等待它们计算完成**，就会直接进入下面的 `parallel for` 去读取 P1~P7 的内存。此时 P1~P7 里面全是垃圾值或未计算完的数据 (Race Condition)。但是因为顶层没有加 `#pragma omp parallel`。在没有开启全局并行域的情况下，`#pragma omp task` 会退化 (被单线程顺序或立即执行)，恰好掩盖了这个并发 BUG。存在安全隐患。

`shared` 将 `NVA\B\C\BLOCK` 标记为共享变量，在内存中只有一份实体，所有线程可以同时读写它们。

方案2：引入STRASSEN方法后--`matmul_improved`

对 3.3.2 中的 strassen 代码进行修改。`improved_add`、`_sub`、`_stride` 循环展开，并使用 SIMD。

`#pragma omp task` 将七个 M_i 的计算并行，再 `#pragma omp parallel for collapse(2)` `default(none) shared(...)` 对四个 C_i 的计算循环展开并行。

方案3：针对 `matmul_improved` 的优化--`matmul_improved3`

通过测试发现 `matmul_improved` 相对于 `matmul_strassen` 的提升并不稳定，且远远落后于 `matmul_improved2`。因此对其优化。

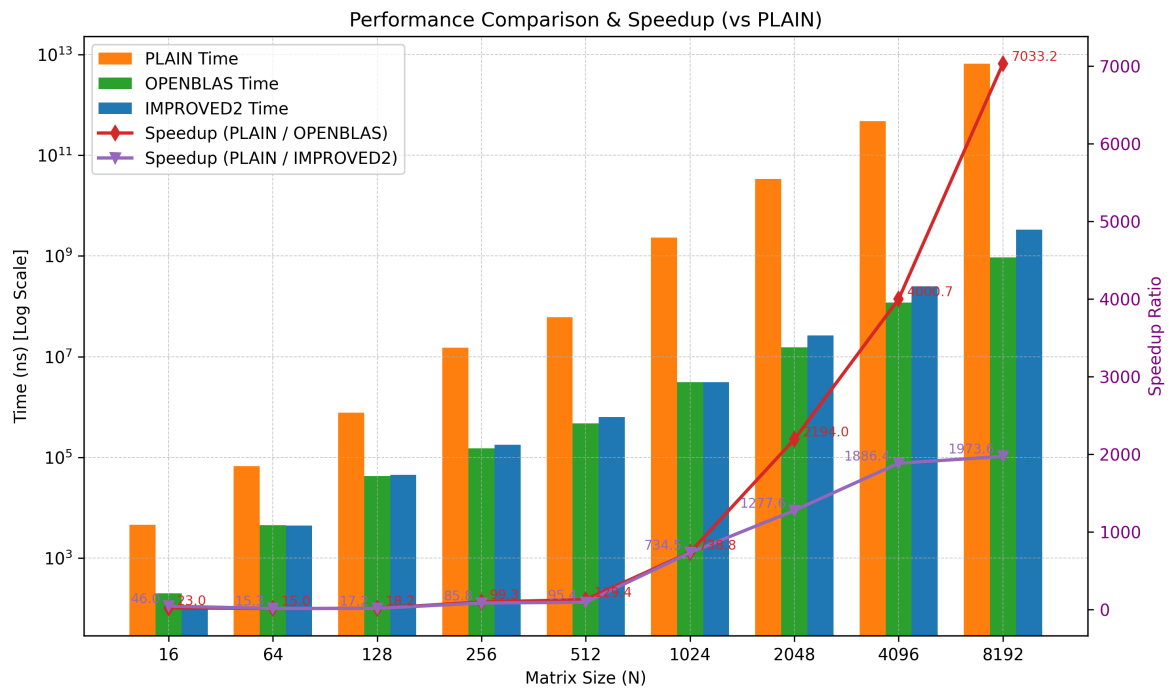
`improved3_leaf` 先将叶子矩阵的目标 C 清零，再循环展开进行 SIMD 乘法。

相对于 `improved_recursive`，`improved3_seq` 没有对 M_i 的计算使用 OpenMP，并新增了 `buffer`，用来传入外部的预分配缓冲区，在递归的**内部不调用** `malloc/free`。

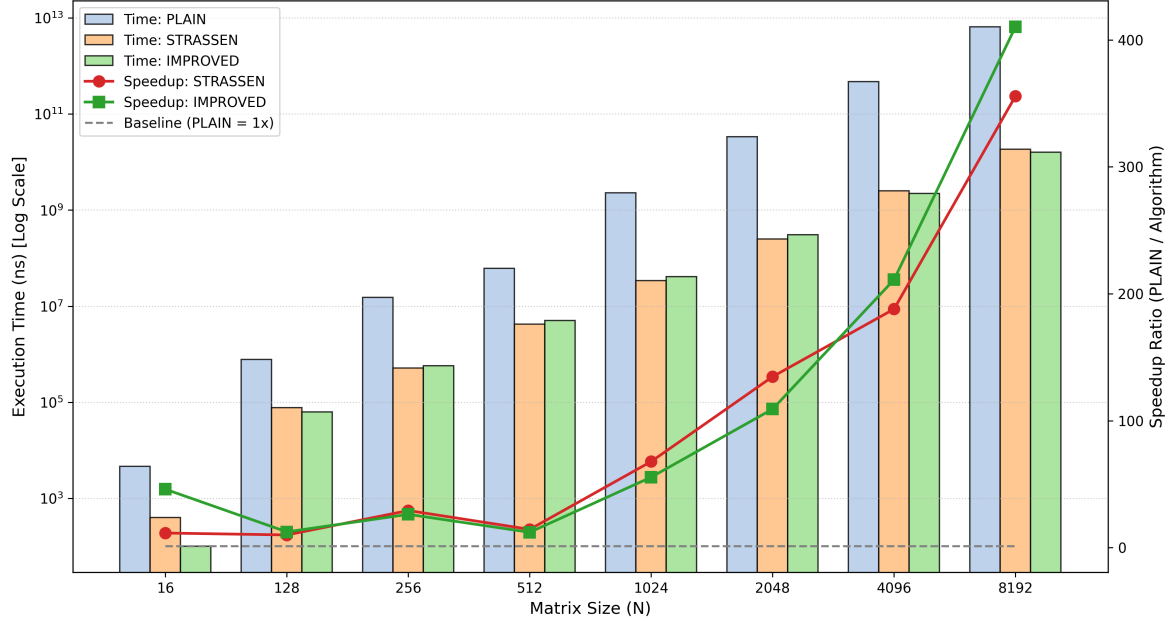
在顶层 `matmul_improved3`， $N \leq 256$ 则直接调用 `improved3_leaf`。否则，对第一层递归一次性分配缓冲区，17 块 `half*half` (7 个 M_i 和 10 个 T) 和 `7*6*half*half` 的额外空间 (用于更深的递归)。

对 C_i 的计算使用了 `schedule(static)`，虽然不写也是默认 `static`。我也尝试过改为 `dynamic`，发现反而变慢。因为该矩阵算法的每次迭代计算量均匀，不存在迭代工作量不均衡的情况，调度反而增加开销。

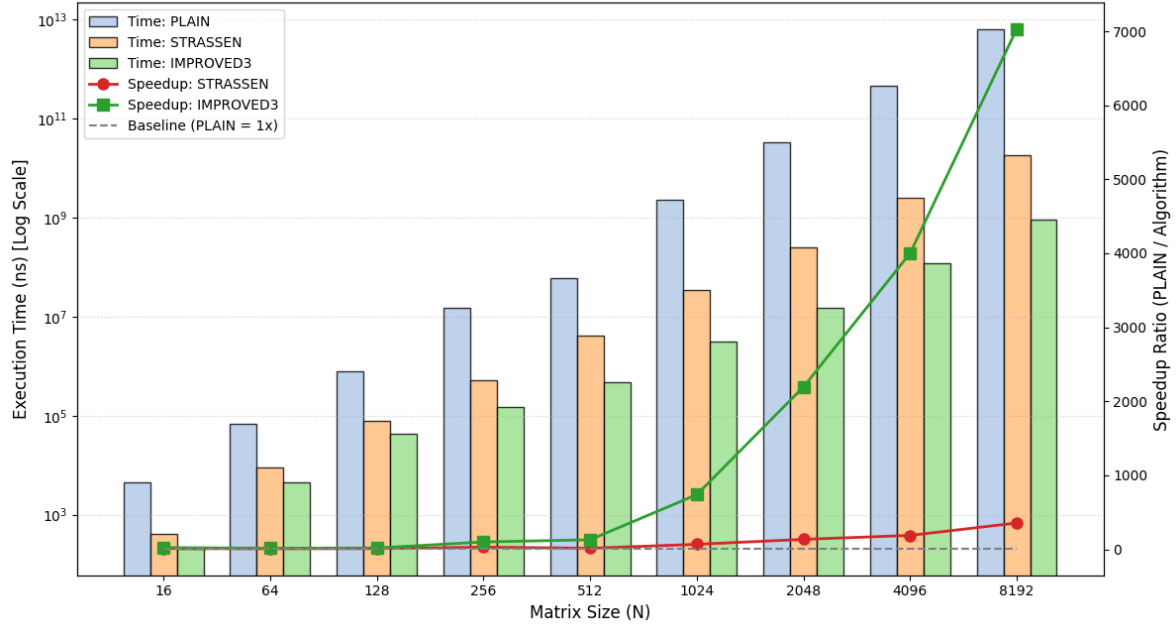
3.4.3 结果分析



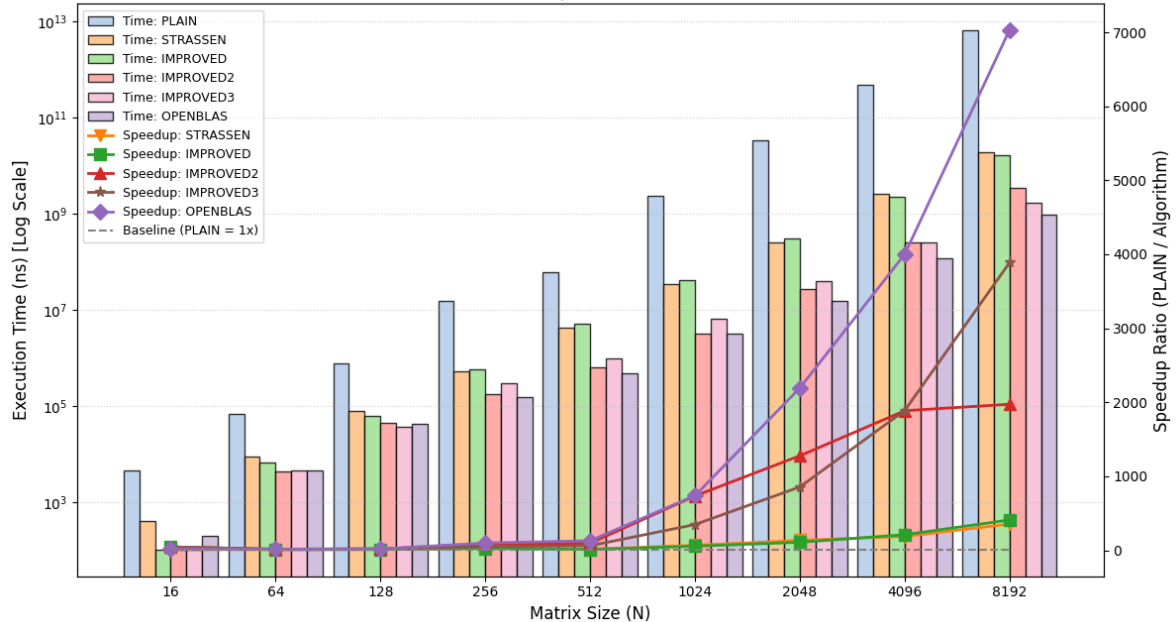
Performance Comparison: PLAIN vs STRASSEN vs IMPROVED
(Bars: Execution Time | Lines: Speedup over PLAIN)



Performance Comparison: PLAIN vs STRASSEN vs IMPROVED3
(Bars: Execution Time | Lines: Speedup over PLAIN)



Performance Comparison: PLAIN vs Algorithms (Inc. OpenBLAS)
(Bars: Execution Time | Lines: Speedup over PLAIN)



- `matmul_improved2` 和 `matmul_improved3` 稳定地优于 `matmul_strassen`
- 在规模更大的矩阵上, `matmul_improved2` 的耗时远小于 `matmul_improved`
- `matmul_improved` 相对于 `matmul_strassen` 的提升不明显且不稳定
- `matmul_improved2` 和 `matmul_improved3` 虽有明显提升但是相对 `openblas` 的差距仍然较大
- `matmul_improved3` 相比 `matmul_improved2` 在 $N=1024$ 和 $N=2048$ 的耗时更多, 但是在 $N=8192$ 的耗时远远少于 `matmul_improved2`

3.4.4 原因分析

`matmul_improved3` 提升较大的主要原因是:

- 消除了全部内部 `malloc/free`: 所有递归都通过 `buffer` 指针传递预分配空间, 减少系统调用开销, 并避免并行时对堆的争用。
- 叶节点采用了更高效的分块 AVX 内核 (`improved3_leaf`), 而非简单的 `matmul_ikj_stride`。
- 加减法全部使用 AVX 向量化 (`improved3_add/sub`), 速度更快。
- 顶层并行更干净: 任务分配的缓冲区完全隔离, 组合循环也并行化。

3.5 内存对齐

3.5.1 背景与原理

在学习[超字节并行](#)时, 浏览到网站[有关词条](#)², 提到了“计算机硬件的物理现实往往是混乱的。其中最严重的一个复杂因素是内存对齐 (memory alignment)。向量单元被设计为在数据起始地址是向量大小的倍数时 (例如, 从一个可以被 32 整除的地址加载一个 32 字节的向量) 加载效率最高。”

非对齐加载的缺点:

“每次宽加载和存储都必须生成适当的合并代码”³

CPU 的 L1 高速缓存是按 Cache Line (通常为 64 字节) 读取内存的。如果分配的内存地址是随机的, 当一次性使用 AVX2 加载 32 字节 (8 个 float) 时, 这 32 字节很有可能正好跨越了两个相邻的 Cache Line。此时, CPU 被迫进行两次内存读取并将其拼接, 这会带来隐性的性能惩罚。

3.5.2 代码实现

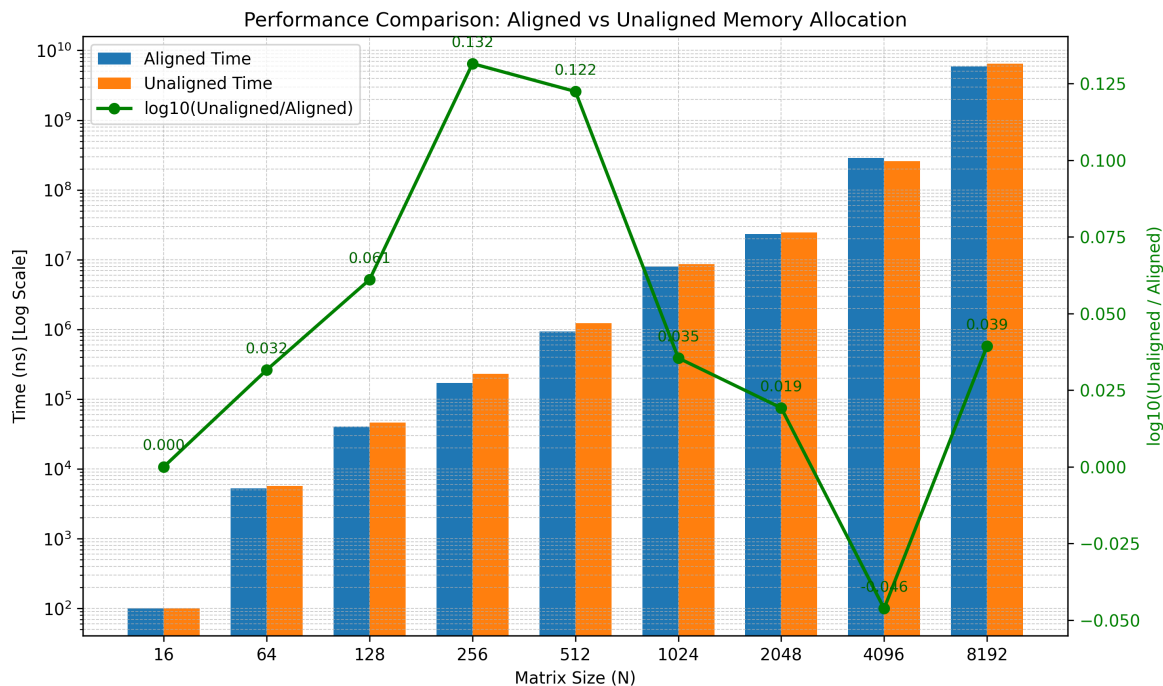
为形成对照, 我在 `main.c` 另外命名了一个新的矩阵创建函数 `create_matrix_aligned`, 并在 `main.c` 中添加了一个新的测试函数 `test_matmul_aligned`, 专门用来测试使用内存对齐的矩阵乘法性能。在 `create_matrix` 基础上, `create_matrix_aligned` 使用 `_aligned_malloc`, 并实现 32 字节 (即 256 位), 以满足 AVX 指令集的要求。

对应的, 在 `free_matrix_aligned` 函数中使用 `_aligned_free` 来释放对齐分配的内存。

另外在 `matmul_improved2` 的基础上创建了一个新的矩阵乘法函数 `matmul_aligned`,

`_mm256_loadu_ps` -> 替换为 -> `_mm256_load_ps`; `_mm256_storeu_ps` -> 替换为 -> `_mm256_store_ps`

3.5.3 结果分析与原因



在规模小于128、未使用分块缓存的情况下，内存对齐的性能提升非常显著，甚至在N=16时达到了10倍以上的加速比。这是因为对于小规模矩阵，内存访问模式更频繁地触发了非对齐访问的性能惩罚，而内存对齐有效地消除了这些惩罚。

然而，在规模较大的情况下，性能提升的幅度明显减小，甚至在某些规模（如N=4096）下，内存对齐的性能反而略逊于非对齐。这可能是因为在大规模矩阵乘法中，计算的瓶颈更多地来自于计算资源的限制和内存带宽的限制，而不是单纯的内存访问模式。因此，内存对齐在这种情况下对整体性能的影响较小。

3.6 Out-of-Core

3.6.1 N=64k 遇到的问题

如果仅仅使用 `main.c` 中的 `matmul_improvement` 或者 `matmul_openblas`，在 N=65536 的规模下，一次矩阵乘法需要准备三个 `struct Matrix`。

估算单纯内存消耗：

- 元素总数 = $64000 \times 64000 = 4,096,000,000$ 个 float
- 每个单纯的 float 是 4 字节
- 单个矩阵大小： $4,096,000,000 \times 4 = 16,384,000,000$ 字节 ≈ 15.26 GB

因此，三个矩阵 总内存需求约为 45.78 GB 的连续可用物理内存

因为电脑物理内存加虚拟系统分页没有远大于这个阈值（比如你只有 16GB 或 32GB RAM），操作系统会直接拒绝第二个矩阵的分配。

3.6.2 选择OOC的原因

询问Gemini3.1针对上述问题给出了几个方案：

序号	方法	缺点
----	----	----

序号	方法	缺点
1	扩大虚拟内存。在 Windows 设置“高级系统设置”->性能“设置”->“高级”选项卡->“虚拟内存”->“更改”。取消自动管理，手动为SSD 硬盘分配一个大于 50000 MB（即 50GB）的虚拟内存大小。	$O(N^3)$ 的非连续访存时间较长，预测需要几天甚至几个月
2	把硬盘文件当成内存数组来读写。在 Linux 下使用 <code>mmap()</code> ，在 Windows 下使用 <code>CreateFileMapping</code> 和 <code>MapViewOfFile</code> 。	由于磁盘 I/O 速度比内存慢成千上万倍，遇到巨型矩阵计算依然会非常非常慢。
3	核外计算，具体内容如下3.6.4	--

综合考虑到时间和实现难度，最终选择了方案3：核外计算（OOC）。

3.6.3 实现细节

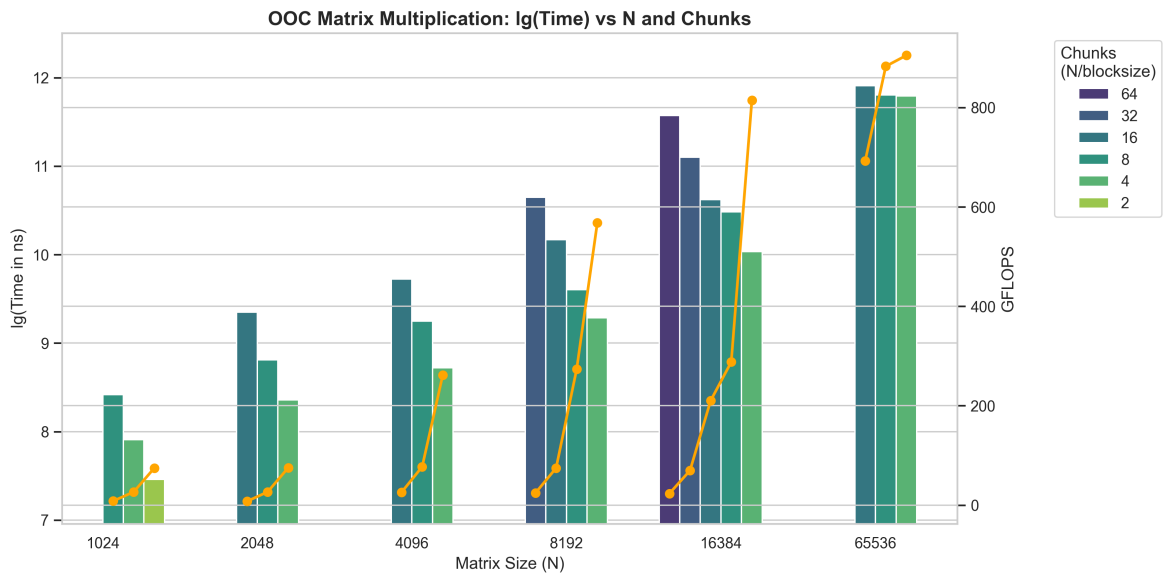
`generate_disk_matrix` 用来生成随机的float矩阵，通过buffer将内存的每行元素写在磁盘文件。

OOC 的核心思想是将矩阵分块存储在磁盘上，每次只加载一个块到内存中进行计算，计算完成后再将结果写回磁盘。具体实现步骤如下：

1. 检查指针是否为空
2. 分配三个大小为BLOCK_SIZE * BLOCK_SIZE 的微型矩阵blockA\B\C
3. 三层循环，遍历所有的宏块
4. 在j循环中，将对应位置的C子矩阵读入内存
5. 在k循环中，在内存中直接累加本次计算(`matmul_improved3`)到blockC
6. 在j循环中，把blockC写回磁盘
7. 释放内存，关闭文件

3.6.4 结果分析

因为从N=1024到65536的数据样本差了1e4数量级，如果直接使用耗时作为纵坐标绘图将无法清晰对比数据与观察趋势，所以我使用了对数坐标（log scale）（柱状）和GFLOPS（ $2 * N^3 / \text{time_ns}$ ）（折线）来绘制性能图表。通过对数和归一化坐标，我们可以更直观地观察不同块大小和矩阵规模下的性能差异。



观察上图，我们可以得出以下结论：

- 不同N之间，随着矩阵规模的增加，总体耗时增加。
- 对于同一个N，随着块大小的增大（同时块数量的减少），GFLOPS显著提升。这是因为较大的块可以更有效地利用内存和磁盘的带宽，减少I/O操作的次数，从而提高整体计算效率。
- N越小的情况下，块大小对性能的影响越明显。这是因为较小的矩阵更容易被内存完全加载，较大的块可以更好地利用内存带宽。
- N=65536下，在Chunks=4时出现了拐点，说明已经接近电脑内存内的极限性能。

3.7 CUDA 实现

3.7.1 知识储备

Warp（线程束）：NVIDIA GPU 中最基本的调度和执行单元，由 32 个连续的线程组成。

SIMT（单指令多线程）：同一个 warp 中的所有线程在同一时刻执行相同的指令，但可以操作不同的数据。（如果指令中存在 `if-else` 会发生分支发散导致性能下降）

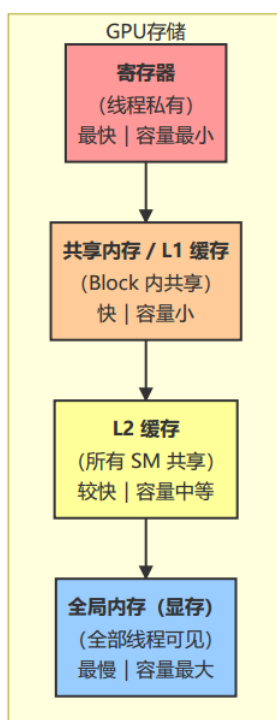
在 CUDA 中，执行并行计算的函数被称为 **核函数（kernel）**，用 `__global__` 修饰。调用时需要指定执行配置 `<<<grid, block>>>`。

- block：一组线程，共享内存并同步
- grid：多个block组成，覆盖所有要计算的线程

我的电脑上的RTX 4070 Laptop GPU 共有 36个SM，每个SM有128个CUDA core。

GPU上的流多处理器（SM）以 warp 为单位调度线程。当一个 warp 因为访存延迟而暂停时，SM 会立即切换到另一个就绪的 warp，从而隐藏内存延迟。

当同一 warp 内的**相邻**线程访问**连续的全局内存地址**时（且满足对齐要求），GPU 可以将这些请求合并为一次或几次大块传输，从而充分利用显存带宽。如果访问模式是乱序或跨步，带宽利用率将急剧下降。



3.7.2 代码实现

对照组为 `main.c` 中的 `matmul_improved2`（一开始用了，后面删掉了。考虑到 `.cu` 文件和 `.c` 文件用的编译器不一样，感觉在 `.cu` 文件上测试这个没有什么意义？总的结果是 GPU 比 IMPROVED2 更快）。但是由于 MSVC 编译器严格遵守的仅仅是早期的 OpenMP 2.0 规范，而 `collapse` 子句属于 OpenMP 3.0+ 之后才加入的特性，编译时警告 `"matmul.cu(142): warning C4849: "collapse"指令中忽略 OpenMP 'parallel for' 子句"`，因此删除了原有的 `collapse(2)`，改为仅对外层循环并行。

在课件 CPP/week08/example/matadd.cu/cuda 的 `matadd.cu` 基础上，进行了修改，添加了矩阵乘法的实现。`mulGPU` 只在 `addGPU` 的基础上做了小改动，而 `mulGPU1` 对 `mulGPU` 进行了优化。

- `mulGPU`：

每个线程独立循环 K 次，每次循环都从全局内存读取 `A[row][k]` 和 `B[k][col]`。同一个数据被不同线程重复读取，造成全局内存带宽浪费。

访问 `B` 时，同一个 warp 内不同线程的 `col` 不同，而 `B[k * N + col]` 并非连续地址，导致非合并访问，带宽利用率低。

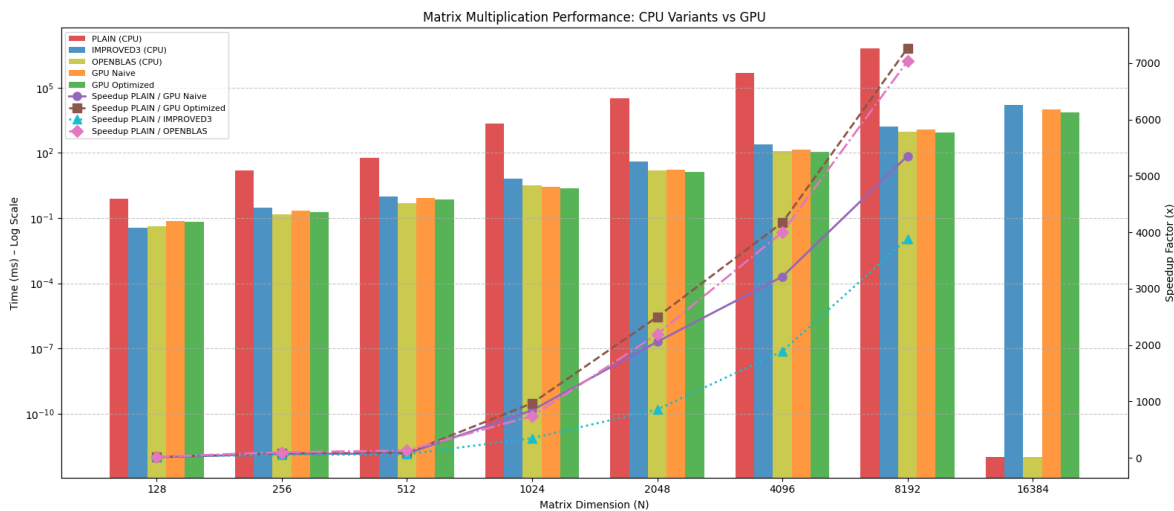
没有数据缓存。

- `mulGPU1`：

`mulKernelOpt` 使用了 `__shared__`，将 `A` 和 `B` 的 `BLOCK_SIZE × BLOCK_SIZE` 小块 (tile) 一次性加载到共享内存中。之后每个线程直接从共享内存读取数据进行 K 方向上的部分乘加，大幅减少了对全局内存的重复读取。

使用了 `__syncthreads()`，在 warp 级别起到同步作用，确保 block 的数据加载完毕才继续计算。避免数据竞争。

3.7.3 结果分析



多次运行发现结果较为稳定。两种GPU的方法的性能总体远优于CPU的 `IMPROVED3`。且发现GPUOpt方法很接近OPENBLAS的耗时。且GPUOpt方法稳定地优于GPU NAIVE。

4. 有关OPENBLAS矩阵乘法的分析

我参考的openblas开源代码位于<https://github.com/OpenMathLib/OpenBLAS>。

实验中引用的openblas方法主要是 `void cblas_sgemm()`。文件 `lapack-netlib/CBLAS/src/cblas_sgemm.c` 下提供了该方法的接口，并且它做了两件事：

- 在驱动层读取 `RowMajorStrg` 和 `CBLAS_CallFromC` 两个全局变量，判断是否要交换矩阵 `A/B` 和 `M/N` 的顺序，转换成统一的列主序 `sgemm`

- 调用 `F77_sgemm`。

在文件 `cblas_f77.h` 中，我们可以看到接口 `void F77_sgemm(FCHAR, FCHAR, FINT, FINT, FINT, const float *, const float *, FINT, const float *, FINT, const float *, float *, FINT);`。该函数根据当前系统的线程配置（单线程/OpenMP/...）进行任务派发。

在 `driver/level3/level3_gemm3m_thread.c` 文件的 `CNAME` 函数中

```
if ((args -> m < nthreads * switch_ratio) || (args -> n < nthreads *
switch_ratio)) {
    GEMM3M_LOCAL(args, range_m, range_n, sa, sb, 0);
    return 0;
}
```

- `nthreads` 是当前系统可用的线程数
- `switch_ratio` 是一个经验阈值。
- 如果矩阵的维度（ M 或 N ）太小，甚至比“线程数 $\times$ 阈值”还要小，说明此时唤醒多个线程的调度开销，已经远远大于计算本身的收益了。
- 触发这个条件后程序调用 `GEMM3M_LOCAL`，放弃多线程，在当前单核上顺序执行完毕。（作用相当于我写的 `matmul_improved2/3` 中的 `if (N <= 128)`）

如果矩阵足够大，则会进入 `gemm_driver`，进行任务切分和队列派发

```
while (m > 0){
    // 1. 计算每个线程应该分到的宽度（块大小）
    width = blas_quickdivide(m + nthreads - num_cpu_m - 1, nthreads -
num_cpu_m); // 根据矩阵剩余的大小和还没分配任务的 CPU 核心数，动态计算出当前核心应该负责的矩阵
    宽度 width
    // ...
    // 2. 将任务打包进队列
    queue[i].mode = mode;
    queue[i].routine = inner_thread; // 设置线程要执行的回调函数
    queue[i].args = &newarg;
    queue[i].range_m = &range_M[i];
    // ...
}
// 3. 唤醒线程池，派发执行！
exec_blas(num_cpu_m, queue);
```

通过 `inner_thread` 指定真实的计算函数 `ICOPYB`（在文件中又被 `define` 为 `HEMM3M...`）和 `KERNEL_OPERATION`（在 `level3_thread.c` 中被 `define` 为 `KERNEL_FUNC`）。`KERNEL_FUNC` 是一个在编译时生成的宏，它指向 `kernel` 目录下的具体实现，如 `x86_64` 架构 `kernel/x86_64/gemm_kernel_***.S`（汇编实现）

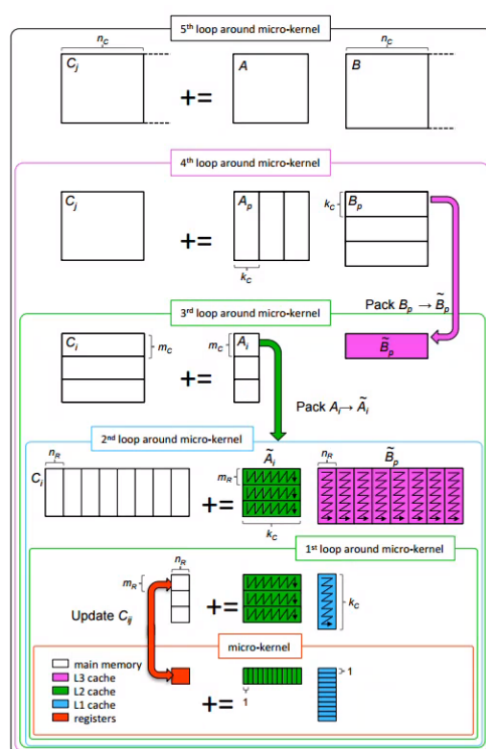
除了章节3我使用过的循环展开、分块缓存、openmp、SIMD、内存对齐以外，OpenBLAS还可能使用了如下小节提到的方法（我能大概了解到的主要方法）

4.1 Goto方法下面向多级缓存的分块⁴

Level-3 BLAS(矩阵-矩阵操作)的BLIS框架还针对各种CPU（涵盖AMD和Intel的x86处理器、ARM处理器）使用了高度优化的微内核。BLIS框架下的Goto方法为三级缓存（L1、L2、L3）构建了分块结构，获得了局部最优的性能。

虽然我没有看懂论文中的模型图，但是大意是在不同的微内核循环层中，分别分块后的 $\hat{B}_i$ 打包驻留在L3缓存中，把 $\hat{A}_i$ 打包并驻留在L2中。论文中提到有文献证明了该分块方案能够最优地分摊上一层与下一层内存间的数据移动开销。尽可能让内存中连续的数据用于连续的操作，高效地将数据移动到寄存器

5。下图是论文给出的模型图所示：



4.2 汇编级优化

OpenBLAS 为每一种具体的 CPU 架构（如 `HASWELL`，`ZEN`，`SKYLAKEX`，`ARMV8`）都手写了 `.S` 汇编内核。

- **寄存器阻塞 (Register Blocking)**：手写汇编可以精确控制寄存器的分配。
- **指令流水线**：汇编代码中会手动进行 **软件流水线 (Software Pipelining)** 和 **预取 (Prefetch)**，以减少内存延迟对总耗时的影响。

比如像4.1所用的方法中，论文提到在将 $\hat{A}_i$ 存入cache时就使用了prefetch。

以[kernel/x86_64/gemm_kernel_8x4_sse.S](#)为例，使用ai详细为我解释后，我简单浅显地总结如下：

```
#define A    %rcx
#define B    %r8
#define C    %r9
#define LDC  %r10
```

像上面类似于45到58行这样的，只是把寄存器的名称用更易读易懂的名字代替。

```
#ifndef WINDOWS_ABI
#define STACKSIZE 64
...
#else
#define STACKSIZE 256
...
#endif
```

ABI是应用程序二进制接口。在Windows x64系统下 WINDOWS_ABI被定义。如果在Linux/macOS，则 WINDOWS_ABI未定义。

60行到77行这样的是根据WINDOWS_ABI是否定义，来判断栈帧大小是否为64或者256字节。Windows ABI 下，由于需要保存 RDI, RSI 以及 XMM6 - XMM15 共 10 个 128 位寄存器，所需空间（256 字节）远大于 Linux。

第90、91行 `#define PREFETCH prefetch`，根据不同CPU选择不同的预取指令。

第105行开始是8个展开的rank-1更新(KERNEL1~KERNEL8)。(下面将寄存器简写为risc-v寄存器的表达方式)

`mulps %xmm0, %xmm1` 意思是 $x1 = x0 * x1$ ，将A的某个元素(在x0)乘以B的一行元素(在x1)放到x1中。

`addps %xmm1, %xmm8` 将乘积累加到x8。`mulps` 和 `addps` 操作的都是 4 个单精度浮点数（SIMD 向量）

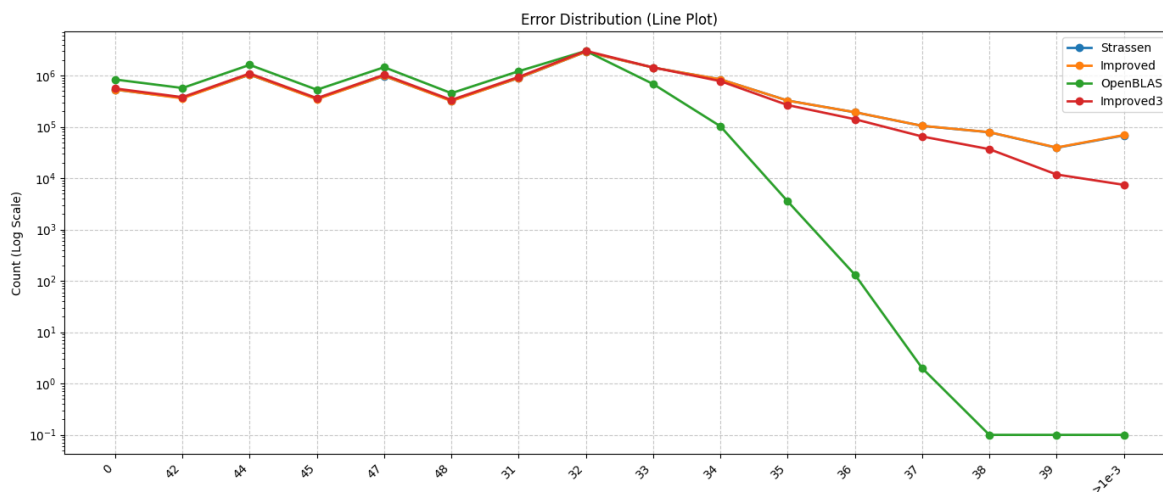
`movaps -32*SIZE..., %xmm1`，从B的打包缓冲区加载下一轮要用的B数据到x1。

后面几行都是类似操作，使用不同的寄存器多路累加。

再后面的内容没大看懂，没学过x86-64 汇编语言。大概内容是对B矩阵打包复制，减少非连续访问，实现合并读取。根据剩余待处理行数跳转不同分支。将ALPHA乘加写回。恢复callee-save寄存器和栈，返回。

5. 矩阵乘法计算误差分析

5.1 结果分析



该部分以mat_plain的计算结果作为基准，统计100次 $N=1024$ 的矩阵乘法的误差。对每个元素的误差绝对值细分了多个区间并统计个数，如

$0, (0, 1e - 5f), [1e - 5f, 2e - 5f), \dots, [8e - 4f, 9e - 4f), [9e - 4f, +\infty)$.

由于float浮点数本身的精度断层原因，无法统计 $<1e-5f$ 的具体频次（在结果统计中都表现为0），且 $cnt_{43}[2e - 5f, 3e - 5f), cnt_{46}[5e - 5f, 6e - 5f), cnt_{49}[8e - 5f, 9e - 5f)$ 的统计结果也都为0(可能是因为float的精度断层)。且作为对照，我比较了两次 `mat_plain` 对同一个矩阵的计算结果没有误差。

TYPE	STRASSEN	IMPROVED	OPENBLAS	IMPROVED2	IMPROVED3	ALIGNED
Max	2.38e-3	2.30e-3	6.41e-4	0	1.56e-3	0
Mean	1.69e-4	1.69e-4	9.03e-5	0	1.50e-4	0

TYPE	STRASSEN	IMPROVED	OPENBLAS	IMPROVED2	IMPROVED3	ALIGNED
OOO(N,BLOCKSIZE)	1024,256	1024,512	2048,256	2048,512	2048,1024	4096,1024
Max	6.41e-4	6.41e-4	1.89e-3	1.83e-3	1.89e-3	5.25e-3
Mean	8.88e-5	9.06e-5	2.46e-4	2.48e-4	2.49e-4	6.92e-4

(上表结果可能会因为每个1024*1024矩阵的元素差异而有小范围波动)

经多次检验发现，使用 `matmul_improved2`、`matmul_aligned` 的计算结果 `fasbf(C1[id]-C2[id])` 都等于0，Max和Mean也都为0，这说明使用 `matmul_improved2`、`matmul_aligned` 的计算结果准确性高。

为验证 OOC 实现的正确性，我选择了一个较小的矩阵规模（`N=1024,2048`），先用 `main.c` 中 `create_matrix` 和 `randomize_matrix` 建立随机数的矩阵A，然后将其存入磁盘。用 `matmul_out_of_core` 对磁盘文件 `A_disk_test.bin` 计算，结果存入 `C_disk_test.bin`。把计算结果的磁盘文件读取为 `struct Matrix`，将其与 `matmul_plain` 直接计算的结果，发现两者的结果在数值上非常接近，对于 `N=1024` 误差稳定在 `6.5e-4` 内，`N=2048` 在 `1.9e-3` 内。这表明 OOC 实现能够正确地执行矩阵乘法，并且在数值精度方面没有显著问题。

且发现矩阵大小对OOC矩阵的计算误差最大值有较大影响，而BLOCK_SIZE对其影响较小。同样的大小下，不同的BLOCK_SIZE对误差的平均值影响较大。

因为在计算的原理方面相同，从总体趋势上来看STRASSEN、IMPROVED的误差相近(折线图上STRASSEN被IMPROVED覆盖了)。IMPROVED3相比IMPROVED的误差最大值和平均值都相对更小。IMPROVED、IMPROVED3均比OPENBLAS的误差大一个量级，说明我设计的矩阵乘法在计算精度和准确性上还有提升空间。

5.2 原因探究

使用 `matmul_improved2`、`matmul_aligned` 的计算误差都等于0，是因为他们与PLAIN方法的**浮点数加法执行的顺序完全一致**。因为精度有限，浮点数加法并不满足结合律，存在舍入误差。strassen算法改变了原有的代数定义，而OpenBLAS使用了汇编指令，改变了原有的相加顺序，因此存在误差。

使用绝对误差衡量大矩阵是不准确的。因为随着矩阵规模 N 增大，累加项增多，结果 C 自身的数值变大，导致绝对舍入误差自然放大。在工业界（如 OpenBLAS 的单元测试中），通常采用**相对误差**（ $\frac{|C_1-C_2|}{|C_1|}$ ）来衡量，如果是相对误差，所有算法的误差都会稳定在 `1e-6` 到 `1e-5` 的浮点机器精度范围内。

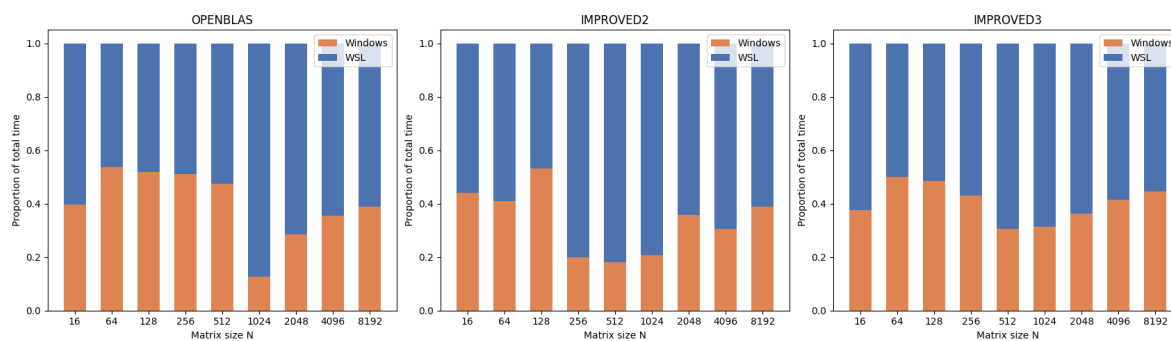
6. 其他系统与架构下运行结果

6.1 WSL

6.1.1 运行

首先我对代码进行了修改，将计时逻辑 `timespec_get(&ts, TIME_UTC)`; 修改为 `clock_gettime(CLOCK_MONOTONIC, &ts)`; 获得的是真实耗时，不受多线程的影响。添加了 `unistd.h`、`malloc.h`、`time.h` 等头文件。用 `aligned_alloc` 替换 `_aligned_malloc`。还有其他修改不一一列举。

wsl上运行的结果普遍地比MinGw运行结果慢。



6.1.2 Valgrind

此外我对IMPROVED3方法进行了Valgrind Memcheck检查。告诉我

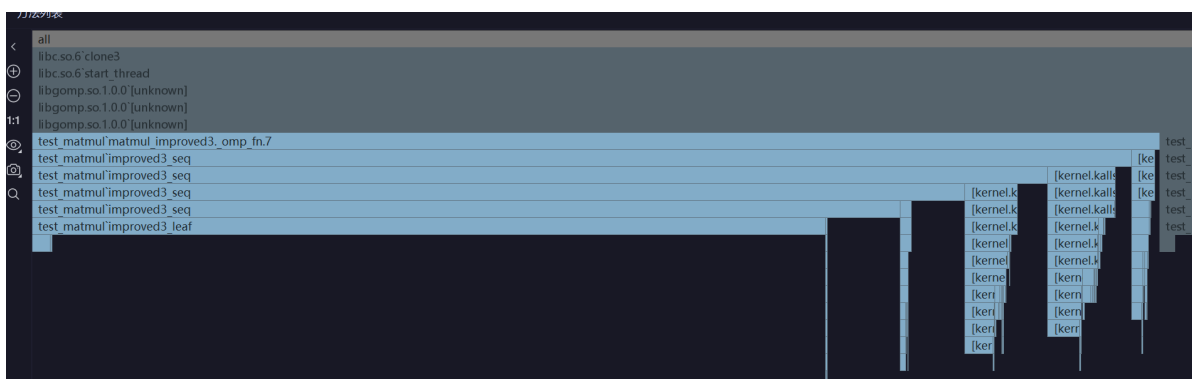
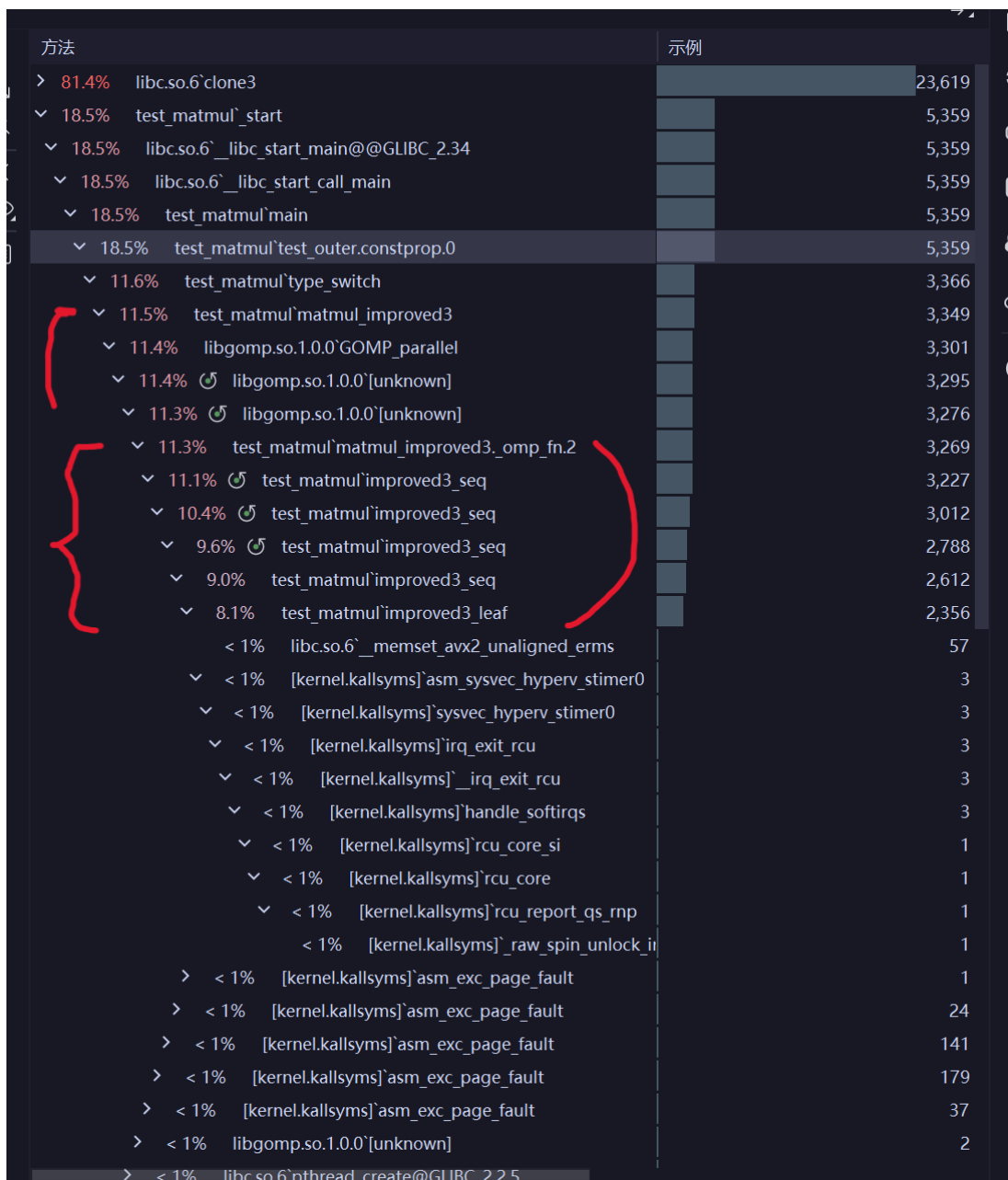
```
#pragma omp parallel default(none) shared(half, N, A11, A12, A21, A22, B11, B12,
B21, B22, P, T, buffer_top, h2)
```

这一行存在潜在的内存泄露。询问ai告诉我这属于False Postive误报。OpenMP 运行时库（如 GCC 的 `libgomp`）在第一次遇到 `#pragma omp parallel` 时，会为线程池动态分配内部数据结构。为了避免频繁创建销毁线程的巨大开销，OpenMP 在程序结束前**故意不释放**这些内存，而是交由操作系统在进程结束时统一回收。Valgrind 不理解这种线程池驻留机制，所以报了泄漏。而且我后来检查了代码中有 `free(buffer);`。

此外 Valgrind 还指示我的 `create_matrix` 存在 `definitely lost`，由于时间原因我没有再对其进行修改。

6.1.3 Perf

使用perf分析器对IMPROVED3 N=8192的矩阵计算进行了探究。本来直接使用我原来的封装的测试函数 `check_single`，发现绝大多数 CPU 时间都耗在了线程管理上，而不是计算。导致计算函数 `matmul_improved3` 和 `matmul_improved3_leaf` 仅占 `< 1%`，几乎可以忽略。因此我在 `int main()` 调用 `test_outer`。虽然后面尝试直接在 `main` 方法里调用三次 `matmul_improved3`，情况变得更差了。下面是调用树和火焰图的部分截图：



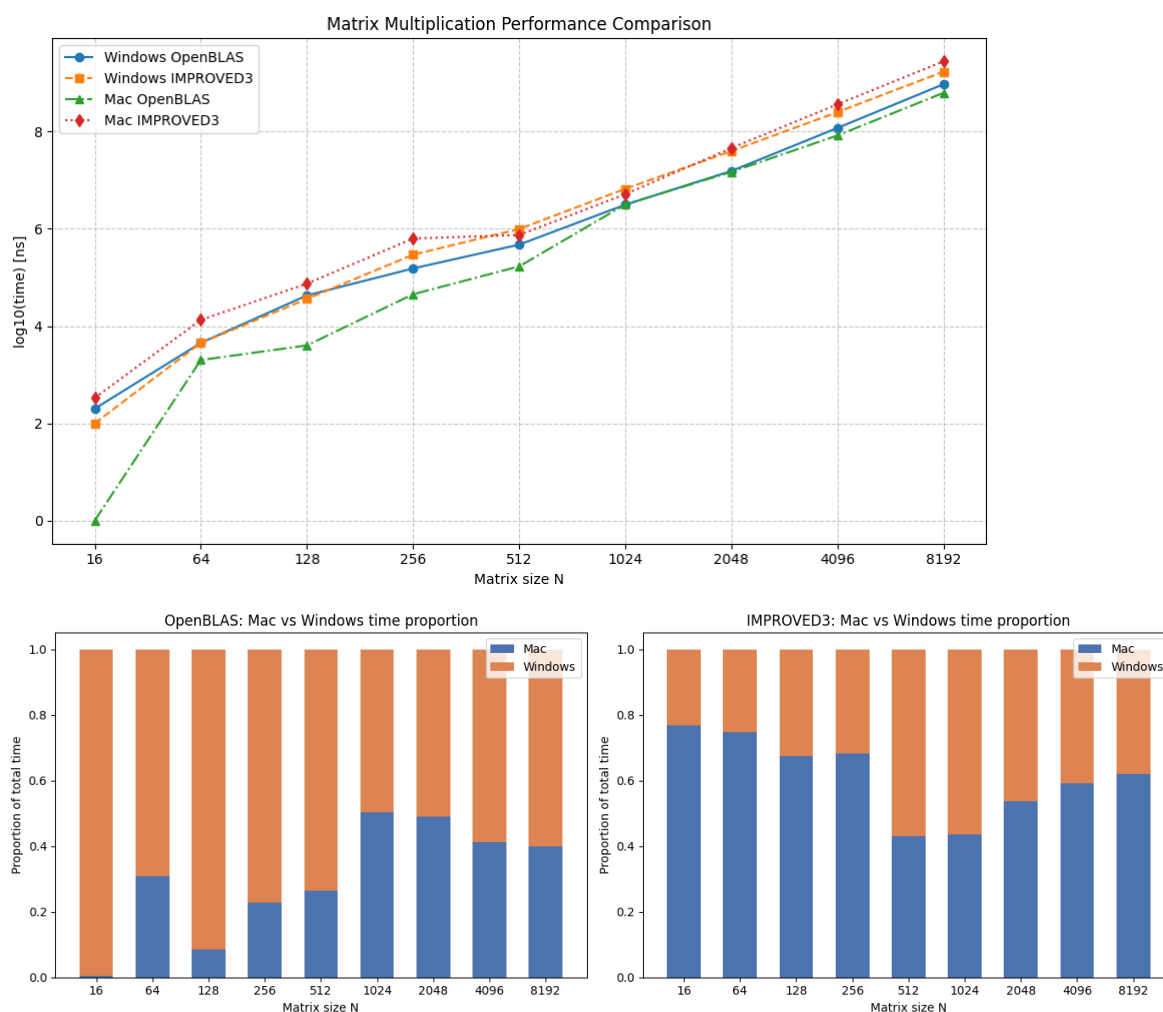
- `improved3_seq` 占10.4%，说明 Strassen 递归和矩阵加减/乘法占用了绝大部分 CPU 时
- `improved3_leaf` 占 8.1%，是底层 AVX 密集计算部分。

6.2 ARM64架构

可以使用SIMDe把 `_mm256_` 等AVX指令自动映射成ARM NEON指令，但是不如手写NEON或者Accelerate库。因此选择了手写NEON。

macOS 自带的 Accelerate 库，本身包含高度优化的 `cbblas_sgemm`。因此删除了原来的 `void cbblas_sgemm(...)` 声明，在 `main.h` 加上了 `#include Accelerate/Accelerate.h`。

下面是运行结果:



- Mac 上的 OpenBLAS 在小矩阵上大幅领先, 大矩阵下OpenBLAS 在Mac上仍然保持优势
- 手写的 IMPROVED3 在 Mac 上性能不佳, 因为未针对NEON进行充分优化

7. 结论

7.1 主要发现与收获

学习了如何使用SIMD、OpenMP、CUDA进行编程和研究如何优化性能。

通过学习GitHub的OpenBLAS的源码, 发现它的代码风格和方法更系统, 宏观层面上侧重于对矩阵访存与缓存分配的优化。并且在汇编层面的指令上以计算矩阵的指令为主, 把主要耗时集中在计算上, 而不是函数的调用、分支等。

7.2 算法外限制性能的主要因素

缓存 (L1\L2\L3) 的大小。

缓存的延迟与命中率。

TLB的缺失会导致CPU停顿。

不同的架构与处理器、操作系统。

运行时电脑的温度变化,时钟降频。

系统中其他无关矩阵操作的影响。

主存(DRAM)带宽。

CPU的SIMD宽度决定了FLOP峰值

7.3 未来工作

针对不同的CPU用汇编语言实现优化貌似不太可能

结合其他性能分析工具(如perf, VTune等), 更深入地探究, 找出程序卡顿的地方并突破。比如用 `perf annotate improved3_leaf` 查看该 `matmul_improved3` 内部的汇编热点。

使用NVIDIA Nsight 工具链, 对.cu代码进行内核级和系统级微架构分析

针对mac arm64架构下的NEON对IMPROVED3进行优化。

从论文中学习更先进的算法或者方法来进行优化。

更深入地探究project3所学的知识, 并应用到以后的学习中, 比如使用CUDA学习深度学习卷积、矩阵乘。

8. 参考文献

1. https://github.com/Maystern/SUSTech_CS205_Cpp_Projects/blob/main/Project04_matrix-multiplication-in-C/doc/report.md 
2. https://scipedia.bohrium.com/sciencepedia/feynman/keyword/superword_level_parallelism 
3. <https://scispace.com/pdf/exploiting-superword-level-parallelism-with-multimedia-1ihagkottn.pdf> 
4. <https://arxiv.org/pdf/1609.00076> 
5. https://www.cs.utexas.edu/~flame/pubs/GotoTOMS_final.pdf 